

Aula 15 — Integração React + .NET API

Módulo: ReactJS + .NET

Carga horária: 4 horas

Projetos base:

- **API:** Aula11.Api (ASP.NET Core)
- **Front:** aula13-react (React + Vite)

Objetivo da aula:

Compreender **CORS**, comparar **Axios vs Fetch**, entender o papel de um **API Gateway** e implementar **integração real React ↔ .NET API** para **listar, cadastrar e editar** recursos.

PARTE 1 — TEORIA (≈ 2 horas)

1 CORS (Cross-Origin Resource Sharing)

◆ O problema

Browsers bloqueiam requisições entre **origens diferentes**:

`http://localhost:5173` → React `https://localhost:5001` → API .NET

◆ A solução

Configurar **CORS no backend**.

Tipos comuns:

- `AllowAnyOrigin` (dev)
- `AllowSpecificOrigins` (prod)

- AllowCredentials

✦ Regra de mercado:

CORS sempre é responsabilidade do backend.

2 Axios vs Fetch

◆ Fetch (nativo)

- ✓ Nativo do browser
- ✗ Boilerplate maior
- ✗ Tratamento manual de erros

```
fetch(url).then(r => r.json());
```

◆ Axios

- ✓ Interceptors
- ✓ JSON automático
- ✓ Tratamento de erros melhor
- ✓ Padrão em projetos grandes

```
axios.get("/produtos");
```

👉 Nesta disciplina: Axios (prepara para JWT e interceptors).

3 API Gateway (Visão Conceitual)

◆ O que é?

Camada intermediária entre Front e Microservices.

Funções:

- Autenticação
- Rate limiting
- Logging
- Versionamento
- Cache

Exemplos:

- YARP (.NET)
- Kong
- NGINX
- Azure API Management

📌 **Aqui:** consumo direto da API

📌 **Em produção:** gateway quase sempre presente.



PARTE 2 — PRÁTICA (≈ 2 horas)



Objetivo Prático

Integrar **React** com **API ASP.NET Core**, implementando:

- Listagem
 - Cadastro
 - Edição
 - Comunicação HTTP real
 - Tratamento básico de erros
-

1 Configurando CORS na API (.NET)

 Program.cs (API)

```
builder.Services.AddCors(options => { options.AddPolicy("ReactPolicy", policy => { policy .WithOrigins("http://localhost:5173")  
.AllowAnyHeader() .AllowAnyMethod(); }); }); app.UseCors("ReactPolicy");
```

 **Importante:** UseCors antes de UseAuthorization .

2 Criando Endpoints CRUD na API

 Controllers/V1/ProdutosController.cs (extensão)

```
[HttpPost] public IActionResult Post(ProdutoDto dto) { var novo = new Produto { Id = _produtos.Max(p => p.Id) + 1, Nome =  
dto.Nome, Preco = dto.Preco, Ativo = true }; _produtos.Add(novo); return CreatedAtAction(nameof(Get), new { id = novo.Id }, dto);  
} [HttpPut("{id}")] public IActionResult Put(int id, ProdutoDto dto) { var produto = _produtos.FirstOrDefault(p => p.Id == id);  
if (produto == null) return NotFound(); produto.Nome = dto.Nome; produto.Preco = dto.Preco; return NoContent(); }
```

3 Instalando Axios no React

```
npm install axios
```

4 Criando Cliente Axios Centralizado

 src/services/api.js

```
import axios from "axios"; const api = axios.create({ baseURL: "https://localhost:5001/api/v1" }); export default api;
```

- ✓ Centraliza URL
 - ✓ Facilita interceptors (JWT – Aula 16)
-

5 Serviço de Produtos (API Layer)

src/services/produtoService.js

```
import api from "../api"; export function listarProdutos() { return api.get("/produtos"); } export function criarProduto(produto) { return api.post("/produtos", produto); } export function atualizarProduto(id, produto) { return api.put(`/produtos/${id}`, produto); }
```

- ✓ Separação de responsabilidades
 - ✓ Código limpo
-

6 Listagem Real no React (useEffect)

components/Produtos.jsx

```
import { useEffect, useState } from "react"; import { listarProdutos } from "../services/produtoService"; import ProdutoForm from "../ProdutoForm"; function Produtos() { const [produtos, setProdutos] = useState([]); const [editando, setEditando] = useState(null); useEffect(() => { carregar(); }, []); async function carregar() { const response = await listarProdutos(); setProdutos(response.data.items ?? response.data); } return ( <div> <ProdutoForm produto={editando} onSalvo={carregar} onCancel={() => setEditando(null)} /> <ul> {produtos.map(p => ( <li key={p.id}> {p.nome} - R$ {p.preco} <button onClick={() => setEditando(p)}>Editar</button> </li> ))} </ul> </div> ); } export default Produtos;
```

7 Cadastro e Edição no Formulário

components/ProdutoForm.jsx

```
import { useEffect, useState } from "react"; import { criarProduto, atualizarProduto } from "../services/produtoService";
function ProdutoForm({ produto, onSalvo, onCancel }) { const [form, setForm] = useState({ nome: "", preco: "" }); useEffect(() =>
{ if (produto) setForm(produto); }, [produto]); function handleChange(e) { setForm({ ...form, [e.target.name]: e.target.value });
} async function submit(e) { e.preventDefault(); if (form.id) { await atualizarProduto(form.id, form); } else { await
criarProduto(form); } setForm({ nome: "", preco: "" }); onSalvo(); } return ( <form onSubmit={submit}> <input name="nome"
placeholder="Nome" value={form.nome} onChange={handleChange} /> <input name="preco" placeholder="Preço" value={form.preco}
onChange={handleChange} /> <button type="submit"> {form.id ? "Atualizar" : "Cadastrar"} </button> {form.id && <button onClick=
{onCancel}>Cancelar</button> } </form> ); } export default ProdutoForm;
```

- ✓ CRUD real
 - ✓ useEffect para edição
 - ✓ Formulário controlado
-

8 Tratamento Básico de Erros

```
try { await criarProduto(form); } catch (error) { alert("Erro ao salvar produto"); }
```

✓ Resultado Esperado

- ✓ React consumindo API .NET real
 - ✓ CORS configurado corretamente
 - ✓ Axios centralizado
 - ✓ Listagem, cadastro e edição funcionando
 - ✓ Arquitetura preparada para JWT
-

Atividade Avaliativa

Desafio prático:

1. Implementar exclusão (DELETE)
2. Adicionar loading durante requisições
3. Tratar erro 400/500
4. Criar arquivo `.env` para URL da API
5. Refatorar para hook `useProdutos`